

Advanced Machine Learning

T.A Merit Samir



Data Preprocessing Revision

Data preprocessing is a critical step in machine learning and data analysis, where **raw data is transformed into a clean**, usable format before feeding it into models. This process helps ensure better model performance and more accurate results.

Common steps involved in data preprocessing include:

1. Data Cleaning

Handling Missing Values: Replace missing values with mean, median, mode, or use techniques like forward fill or backward fill. Alternatively, rows or columns with too many missing values can be removed.



Data Preprocessing Revision

1. Data Cleaning

Outlier Detection and Removal: Use statistical methods like Z-scores or IQR to detect and potentially remove outliers.

Fixing Errors: Ensure data integrity by correcting formatting issues, removing duplicates, and standardizing units or formats.

2. Data Transformation

Normalization/Scaling: Scale features to a specific range (e.g., 0 to 1) or normalize them to make the data comparable across features. Popular techniques include Min-Max scaling and Z-score normalization.



Data Preprocessing Revision

2. Data Transformation

Log Transformation: Used to handle skewed data by transforming distributions closer to normal distribution.

Encoding Categorical Data:

One-Hot Encoding: Convert categorical variables into binary vectors.

Label Encoding: Assign unique integer values to categories.

Color			
Red			
Red			
Yellow			
Green			
Yellow			



Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

One-Hot Encoding

color	
red	
green	
blue	
red	



color
0
1
2
0

Label Encoding

Data Preprocessing Revision

3. Data Reduction

Dimensionality Reduction: Reduce the number of features using techniques like Principal Component Analysis (PCA) or Linear Discriminant Analysis (LDA).

Feature Selection: Use statistical methods to retain only the most important features (e.g., Chi-Square test, recursive feature elimination).

4. Data Splitting

Train-Test Split: Split data into training and testing sets, usually in an 80-20 or 70-30 ratio, to evaluate model performance.

Cross-Validation: Further divides data into folds for cross-validation to ensure robustness.



Data Preprocessing Revision

5. Feature Engineering

Creating New Features: Based on domain knowledge, new features can be created to make the dataset more informative.

Binning: Transform continuous variables into categories (e.g., age ranges).

6. Handling Imbalanced Data

Resampling: Oversample the minority class or undersample the majority class.

SMOTE (Synthetic Minority Over-sampling Technique): Generate synthetic samples for the minority class.

These preprocessing techniques help prepare the data in a way that maximizes the efficiency and accuracy of machine learning algorithms.

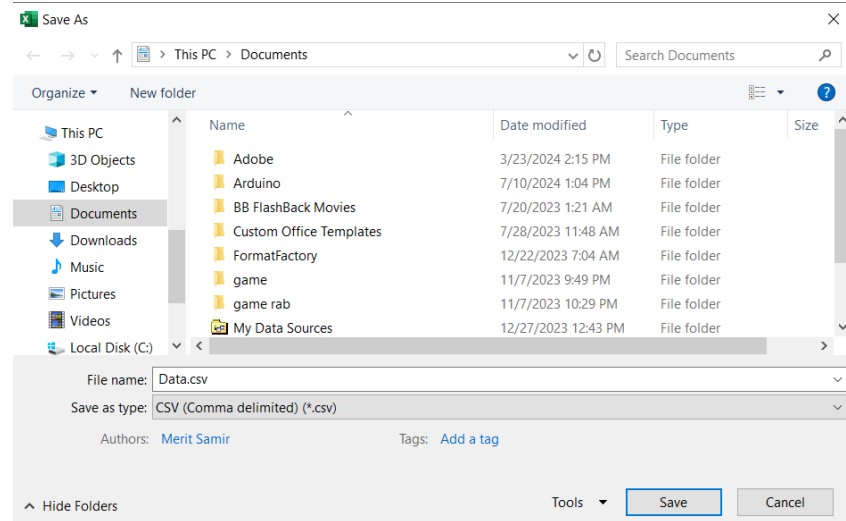


Data Preprocessing

To perform data preprocessing, you first need a dataset. After loading the dataset, you can apply various preprocessing steps.

- Create a simple dataset and save it as CSV or XLSX file

	A	B	C	D	E
1	Country	Age	Salary	Purchased	
2	France	44	72000	No	
3	Spain	27	48000	Yes	
4	Germany	30	54000	No	
5	Spain	38	61000	No	
6	Germany	40		Yes	
7	France	35	58000	Yes	
8	Spain		52000	No	
9	France	48	79000	Yes	
10	Germany	50	83000	No	
11	France	37	67000	Yes	
12					
13					



Load dataset

-Open colab notebook or jupyter notebook and load data

Importing the libraries

```
In [4]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the dataset

```
In [13]: dataset = pd.read_csv('Data.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

1. Loading the Dataset:from the Data.csv file into a Pandas DataFrame called dataset.

2. Selecting Features and Target:

X: Contains all columns except the last one.

y: Contains only the last column, which is assumed to be the target variable.

This code selects columns and stores them in X and y as NumPy arrays.



Load dataset

In [14]:

```
print(X)
```

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

In [7]:

```
print(y)
```

```
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Taking care of missing data

```
In [15]: from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.transform(X[:, 1:3])
```

```
In [16]: print(X)

[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

- **SimpleImputer** class from Scikit-learn, which is used for handling missing values in datasets.
- **imputer.fit(X[:, 1:3])**: This step calculates the mean for the columns in the X matrix where the imputer is applied.
- **imputer.transform(X[:, 1:3])**: This replaces any NaN values in the selected columns (1 and 2) with the calculated mean values.
- The **transformed data** is then reassigned back to X[:, 1:3], updating the original matrix.



Taking care of missing data

```
In [16]: print(X)
```

```
[[ 'France' 44.0 72000.0]
 [ 'Spain' 27.0 48000.0]
 [ 'Germany' 30.0 54000.0]
 [ 'Spain' 38.0 61000.0]
 [ 'Germany' 40.0 63777.77777777778]
 [ 'France' 35.0 58000.0]
 [ 'Spain' 38.77777777777778 52000.0]
 [ 'France' 48.0 79000.0]
 [ 'Germany' 50.0 83000.0]
 [ 'France' 37.0 67000.0]]
```



Encoding categorical data

Encoding the Independent Variable

```
In [17]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder='passthrough')
X = np.array(ct.fit_transform(X))
```

- This code is performing one-hot encoding on a categorical feature in dataset using ColumnTransformer from Scikit-learn.
- ColumnTransformer: This helps apply different preprocessing steps to specific columns in a dataset.
- OneHotEncoder: This is used to convert categorical data into a format that can be provided to ML algorithms (one-hot encoding).
- transformers=[('encoder', OneHotEncoder(), [0])]:
- 'encoder': This is just a name for the transformation.
- OneHotEncoder(): The transformer to apply (one-hot encoding in this case).
- [0]: The index of the column to be transformed, meaning the first column (index 0) of X will be one-hot encoded.
- remainder='passthrough': This ensures that columns that are not transformed (i.e., those not specified in the transformers list) are passed through the transformer unchanged.



Encoding categorical data

```
In [18]: print(X)
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

Encoding the Dependent Variable

```
In [19]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
```

```
In [20]: print(y)
[0 1 0 0 1 1 0 1 0 1]
```

- This code is using **Label Encoding** to convert categorical labels into numeric form, which is typically used when the target variable (y) contains categorical values.



Splitting the dataset into the training and test set

```
In [21]: from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 1)
```

```
In [22]: print(X_train)  
[[0.0 0.0 1.0 38.77777777777778 52000.0]  
 [0.0 1.0 0.0 40.0 63777.77777777778]  
 [1.0 0.0 0.0 44.0 72000.0]  
 [0.0 0.0 1.0 38.0 61000.0]  
 [0.0 0.0 1.0 27.0 48000.0]  
 [1.0 0.0 0.0 48.0 79000.0]  
 [0.0 1.0 0.0 50.0 83000.0]  
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
In [23]: print(X_test)  
[[0.0 1.0 0.0 30.0 54000.0]  
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
In [24]: print(y_train)  
[0 1 0 0 1 1 0 1]
```

```
In [25]: print(y_test)  
[0 1]
```

Feature Scaling

```
In [26]: from sklearn.preprocessing import StandardScaler  
sc = StandardScaler()  
X_train = sc.fit_transform(X_train)  
X_test = sc.transform(X_test)
```

- This code is using **Standardization** to scale the features in training and test datasets (X_train and X_test).
- Imports the StandardScaler class from Scikit-learn, which is used for standardizing features by removing the mean and scaling to unit variance.
- fit_transform(X_train): This does two things:
- fit(X_train): It calculates the mean and standard deviation of each feature (column).
- transform(X_train): It then uses the calculated mean and standard deviation to scale (standardize) the features in X_train such that each feature has a mean of 0 and a standard deviation of 1.

This ensures that the training data is centered and scaled.



Feature Scaling

In [27]: `print(X_train)`

```
[[-0.77459667 -0.57735027  1.29099445 -0.19159184 -1.07812594]
 [-0.77459667  1.73205081 -0.77459667 -0.01411729 -0.07013168]
 [ 1.29099445 -0.57735027 -0.77459667  0.56670851  0.63356243]
 [-0.77459667 -0.57735027  1.29099445 -0.30453019 -0.30786617]
 [-0.77459667 -0.57735027  1.29099445 -1.90180114 -1.42046362]
 [ 1.29099445 -0.57735027 -0.77459667  1.14753431  1.23265336]
 [-0.77459667  1.73205081 -0.77459667  1.43794721  1.57499104]
 [ 1.29099445 -0.57735027 -0.77459667 -0.74014954 -0.56461943]]
```

In [28]: `print(X_test)`

```
[[-0.77459667  1.73205081 -0.77459667 -1.46618179 -0.9069571 ]
 [ 1.29099445 -0.57735027 -0.77459667 -0.44973664  0.20564034]]
```



Why Use Standardization?

• **Standardization** transforms the data so that it has a mean of 0 and a standard deviation of 1. It helps many machine learning algorithms (like SVM, logistic regression, neural networks) to perform better, especially when the data has different ranges or scales (e.g., age vs. salary).

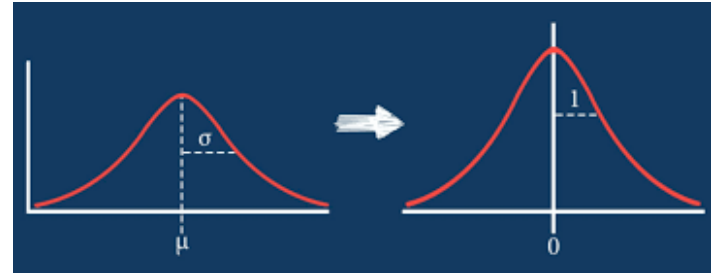
$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

standardization

$$x_{norm}^{(i)} = \frac{x^{(i)} - x_{min}}{x_{max} - x_{min}}$$

min-max scaling
("normalization")

	input	standardized	normalized
0	0	-1.46385	0.0
1	1	-0.87831	0.2
2	2	-0.29277	0.4
3	3	0.29277	0.6
4	4	0.87831	0.8
5	5	1.46385	1.0





merit.samir@nmu.edu.eg

Thank you

